

AI-Optimized Gateway Proxy

Diseño, fundamentos de concurrencia y arquitectura de un Gateway de IA de alta disponibilidad en Go

Documento técnico de arquitectura y aprendizaje

Septiembre de 2026

Resumen

Este documento define, desde primeros principios, los conceptos, decisiones arquitectónicas y mecanismos de bajo nivel necesarios para construir `ai-optimized-gateway-proxy`, un gateway concurrente y de alto rendimiento para enrutar peticiones hacia modelos de lenguaje locales y proveedores externos. El objetivo no es únicamente producir un servicio funcional, sino comprender por qué determinadas decisiones de concurrencia, memoria, red, tolerancia a fallos, caching y observabilidad son correctas o incorrectas bajo carga real. El sistema final combinará un backend en Go, un dashboard en React + TypeScript, balanceo de carga, health checking, failover, rate limiting, caché LRU concurrente, métricas en tiempo real y herramientas de profiling y benchmarking.

Índice

1. Introducción y objetivos	2
2. Arquitectura objetivo del sistema	2
2.1. Vista de alto nivel	2
2.2. Responsabilidades principales	3
3. Modelo de ejecución de Go	3
3.1. Concurrencia frente a paralelismo	3
3.2. Goroutines, threads y scheduler G-M-P	3
3.3. Stacks, heap y escape analysis	4
4. Primitivas de concurrencia y modelo de memoria	5
4.1. Channels	5
4.2. Mutex, RWMutex y atomics	5
4.3. Race conditions, data races y happens-before	6
5. Worker pools, bounded concurrency y backpressure	6
5.1. Por qué no basta con crear goroutines	6
5.2. Bounded concurrency	6
5.3. Backpressure	6
5.4. CPU-bound frente a I/O-bound	7
6. Context, cancelación y deadlines	7
7. Servidor HTTP y networking	7
7.1. Ciclo request/response	7
7.2. Timeouts críticos	8
7.3. Cliente HTTP y connection pooling	8

8. Rate limiting desde cero	8
8.1. Rate limiting vs concurrency limiting	8
8.2. Token Bucket	8
8.3. Refill lazy frente a ticker	9
8.4. Claves del limiter	9
9. Cache In-Memory y LRU concurrente	9
9.1. Por qué una LRU usa mapa + lista doble	9
9.2. Concurrencia y locks	10
9.3. TTL y tamaño	10
9.4. Cache key para LLM	10
9.5. Singleflight y cache stampede	10
10. Routing, load balancing y health checking	11
10.1. Round Robin	11
10.2. Algoritmos comparados	11
10.3. Health checks activos y pasivos	11
11. Failover, retries, circuit breaker y bulkheads	12
11.1. Partial failure	12
11.2. Retries	12
11.3. Circuit Breaker	12
11.4. Bulkheads	12
12. Streaming, SSE y WebSockets	13
12.1. Streaming de respuestas LLM	13
12.2. SSE vs WebSocket	13
13. Pub/Sub interno y slow consumers	13
14. Métricas y observabilidad	14
14.1. Tipos de métricas	14
14.2. Percentiles y tail latency	14
14.3. Métricas del gateway	14
14.4. Logs, métricas y traces	14
15. Memoria, GC y optimización	15
15.1. Hot path	15
15.2. sync.Pool	15
15.3. False sharing	15
15.4. Profiling	15
16. Little's Law, throughput y saturación	15
17. Seguridad y límites defensivos	16
18. Frontend React y dashboard en tiempo real	16
18.1. Paneles recomendados	16
18.2. Evitar re-renders excesivos	17
19. Estructura recomendada del monorepo	17
19.1. Por qué esta estructura	18

20.Flujo completo de una request	18
21.Ejemplo práctico integrado	19
22.Testing y verificación de corrección	20
22.1. Unit tests	20
22.2. Race detector	20
22.3. Load testing	20
23.CI/CD y despliegue	21
24.Roadmap de aprendizaje por milestones	21
25.Errores de diseño que el proyecto debe evitar	22
26.Criterios de calidad de nivel Mid/Senior	22
27.Conclusión	22

1. Introducción y objetivos

ai-optimized-gateway-proxy se plantea como un proyecto de portafolio de nivel Mid/Senior con dos objetivos simultáneos:

1. Construir un sistema útil: un gateway capaz de recibir tráfico de aplicaciones cliente, aplicar políticas de admisión, seleccionar un proveedor LLM, reutilizar respuestas cacheables, degradarse de forma controlada ante fallos y exponer telemetría en tiempo real.
2. Desarrollar criterio de ingeniería: comprender cómo interactúan Go, el scheduler, el sistema operativo, la memoria, las conexiones TCP, las colas, los locks, las cachés y los proveedores externos bajo concurrencia.

El proyecto prioriza una idea esencial: **primero corrección, después concurrencia, finalmente rendimiento**. Optimizar prematuramente un diseño incorrecto crea sistemas rápidos en condiciones ideales y frágiles bajo condiciones reales.

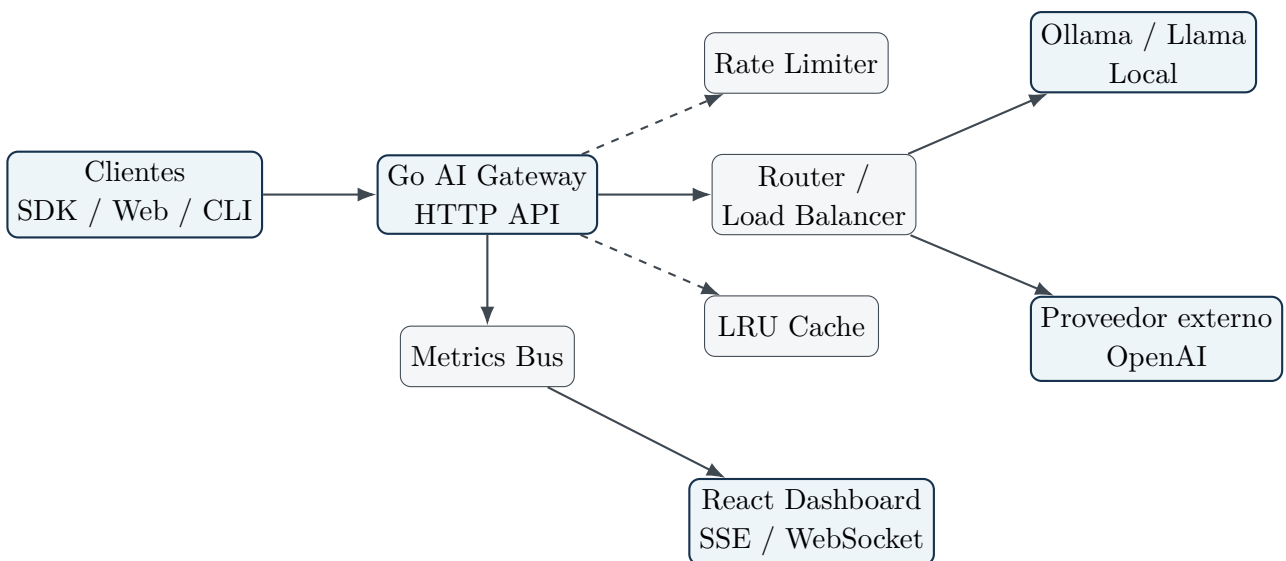
Principio rector

Un sistema de alto rendimiento no es el que procesa una request muy rápido de forma aislada, sino el que mantiene latencia, throughput, estabilidad y uso de recursos dentro de límites razonables cuando aumenta la carga o cuando partes de la infraestructura fallan.

2. Arquitectura objetivo del sistema

El gateway actuará como una capa de control entre los clientes y múltiples proveedores de inferencia. Su responsabilidad excede la de un reverse proxy convencional: decidirá *si* una request puede entrar, *cómo* se procesa, *qué proveedor* debe atenderla, *cuándo* debe reintentarse, *si* puede resolverse desde caché y *qué métricas* deben generarse.

2.1. Vista de alto nivel



Componente	Responsabilidad	Riesgo principal
HTTP Server	Aceptar conexiones y modelar el ciclo request/response	Timeouts incorrectos, cuerpos sin límite, fugas de goroutines
Rate Limiter	Controlar frecuencia y ráfagas	Contención y crecimiento ilimitado de buckets
Worker Pool / Admission	Limitar trabajo en vuelo	Colas sin límite y aumento explosivo de latencia
Cache LRU	Reutilizar respuestas y reducir coste	Locks calientes, memoria no acotada, invalidación incorrecta
Router	Seleccionar proveedor	Distribución injusta o routing sin conocimiento de salud
Health Checker	Detectar estado de backends	Flapping, falsos positivos y falsos negativos
Failover / Retry	Recuperarse de fallos parciales	Retry storms y duplicación de trabajo
Metrics Bus	Desacoplar generación y consumo de métricas	Slow consumers y backpressure
Dashboard	Exponer estado operativo	Re-renderizado excesivo y pérdida de eventos

2.2. Responsabilidades principales

3. Modelo de ejecución de Go

3.1. Concurrencia frente a paralelismo

Concurrencia significa estructurar un programa para que múltiples tareas puedan progresar de forma solapada. **Paralelismo** significa que varias tareas ejecutan literalmente al mismo tiempo en diferentes cores.

Un gateway de IA es predominantemente I/O-bound: una goroutine puede pasar gran parte de su vida esperando red, TLS, DNS, un socket o la respuesta del modelo. En ese escenario, Go puede mantener muchas requests en progreso con un número relativamente pequeño de threads del sistema operativo.

3.2. Goroutines, threads y scheduler G-M-P

Go separa tres conceptos internos:

- **G (Goroutine):** unidad lógica de ejecución gestionada por el runtime de Go. Una goroutine no es un thread del sistema operativo.
- **M (Machine):** thread real del sistema operativo sobre el que termina ejecutándose código Go.
- **P (Processor):** contexto lógico del scheduler de Go. Mantiene los recursos necesarios para ejecutar goroutines y se asocia temporalmente con un M. Un M necesita un P para ejecutar código Go.

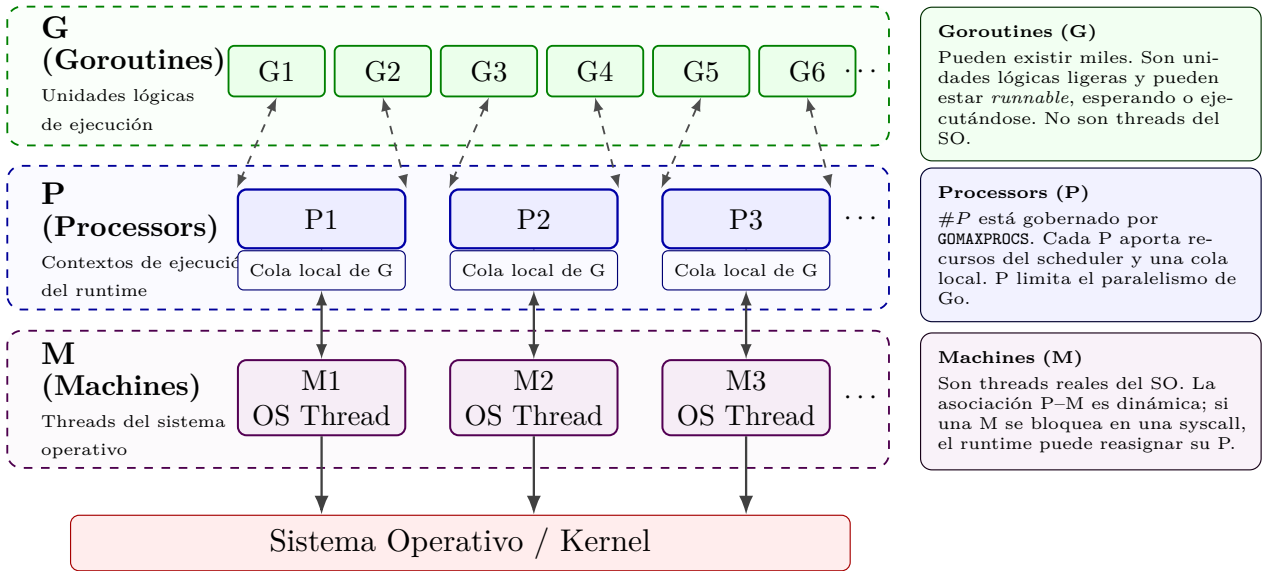


Figura 1: Modelo de ejecución G-M-P del runtime de Go. Las goroutines se multiplexan sobre contextos P, que se asocian dinámicamente con threads M del sistema operativo.

Entidad	Qué representa	Idea clave
G (Goroutine)	Unidad lógica de ejecución gestionada por el runtime; ligera y con stack dinámico.	Se multiplexa sobre P; no equivale a un OS thread. Puede estar <i>runnable</i> , esperando o bloqueada por I/O/sincronización.
P (Processor)	Contexto lógico del runtime que contiene recursos para ejecutar código Go: cola local, cachés y estado del scheduler.	El número de P limita el paralelismo de código Go y está gobernado por GOMAXPROCS.
M (Machine)	Thread real del sistema operativo utilizado por el runtime para ejecutar código Go y realizar operaciones que llegan al kernel.	Ejecuta goroutines cuando está asociado a un P. La asociación P-M es dinámica y puede cambiar ante bloqueos o decisiones del scheduler.

El modelo no debe interpretarse como una asociación permanente $G \rightarrow P \rightarrow M$. Las goroutines pueden cambiar de P a lo largo del tiempo y los P pueden asociarse a distintos M. Si un M queda bloqueado en una syscall, el runtime puede desacoplar su P y entregárselo a otro M para seguir ejecutando goroutines. Esta multiplexación es una de las razones por las que Go puede mantener un número de goroutines muy superior al número de threads activos del sistema operativo.

GOMAXPROCS controla cuántos P pueden ejecutar código Go simultáneamente. Esto no limita el número de goroutines existentes; limita el paralelismo de ejecución de Go sobre CPU.

Aplicación al gateway

Crear una goroutine por request no es necesariamente peligroso por el coste de la goroutine en sí. El peligro real está en los recursos transitivos: memoria de buffers, conexiones upstream, colas, objetos de request, respuestas y trabajo que se acumula cuando el backend LLM procesa más lento de lo que llega el tráfico.

3.3. Stacks, heap y escape analysis

Cada goroutine comienza con un stack pequeño que puede crecer dinámicamente. El compilador intenta mantener variables locales en stack cuando su vida está acotada. Si una referencia debe

sobrevivir al frame actual, o si el compilador no puede demostrar que es segura en stack, el valor puede *escapar* al heap.

El heap es gestionado por el garbage collector. Por tanto, en rutas calientes del gateway, muchas asignaciones efímeras pueden traducirse en más trabajo del GC.

Ejemplos de potencial presión de memoria:

- Leer cuerpos HTTP completos en memoria cuando podrían procesarse como stream.
- Crear nuevos buffers para cada transformación JSON.
- Copiar slices y strings innecesariamente.
- Mantener respuestas grandes en caché sin límites de tamaño.

4. Primitivas de concurrencia y modelo de memoria

4.1. Channels

Los channels permiten pasar ownership o coordinar trabajo entre goroutines. Un channel sin buffer sincroniza emisor y receptor; un channel con buffer desacopla parcialmente ambos lados.

```
1 type Job struct {  
2     RequestID string  
3     Payload []byte  
4 }  
5  
6 jobs := make(chan Job, 256)
```

Listing 1: Canal con buffer para una cola de trabajos limitada

El tamaño 256 no es un detalle cosmético: define cuánto trabajo puede acumularse antes de que el productor quede bloqueado o deba aplicar una política de rechazo.

4.2. Mutex, RWMutex y atomics

Un `sync.Mutex` garantiza exclusión mutua. Un `sync.RWMutex` permite múltiples lectores simultáneos y un único escritor, pero introduce mayor complejidad interna. Solo compensa cuando las lecturas dominan ampliamente y las secciones críticas no son tan pequeñas que el overhead del `RWMutex` supere el beneficio.

Los atomics son adecuados para estados simples como contadores o índices round-robin. No sustituyen un lock cuando existen invariantes sobre múltiples variables.

Primitiva	Úsala cuando	Evítala cuando
Channel	Hay flujo de trabajo, ownership o señalización entre goroutines	Solo necesitas proteger una pequeña estructura compartida
Mutex	Hay invariantes sobre uno o varios campos	Solo quieres incrementar un contador independiente
RWMutex	Lecturas muy frecuentes, escrituras escasas y mediciones demuestran beneficio	La sección crítica es diminuta o las escrituras son frecuentes
Atomic	Estado escalar y operación indivisible	Debes mantener coherencia entre varios campos
WaitGroup	Necesitas esperar la finalización de un conjunto conocido de goroutines	Necesitas cancelar o propagar errores por sí solo

4.3. Race conditions, data races y happens-before

Una **data race** ocurre cuando dos goroutines acceden concurrentemente a la misma ubicación de memoria, al menos una escritura, sin sincronización suficiente. Una **race condition** es más general: el resultado lógico depende del orden de eventos, aunque no exista una data race detectable.

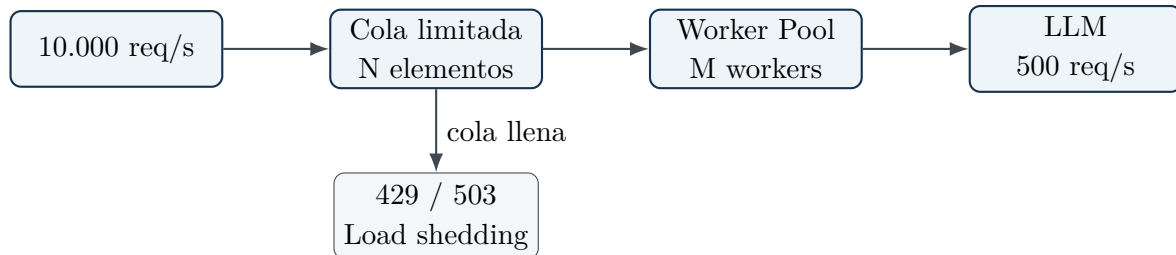
Ejemplo: dos requests consultan la caché al mismo tiempo, ambas observan un miss y ambas llaman a OpenAI. El código puede estar libre de data races, pero existe una condición de carrera a nivel de comportamiento: se duplica una operación costosa.

La solución conceptual puede ser *singleflight* o deduplicación de requests en vuelo.

5. Worker pools, bounded concurrency y backpressure

5.1. Por qué no basta con crear goroutines

Supongamos que el gateway recibe 10.000 requests/s, pero los proveedores pueden completar 500 requests/s de forma sostenible. Si aceptamos trabajo ilimitado, la cola efectiva se desplaza a memoria y el tiempo de espera crece sin límite.



5.2. Bounded concurrency

Limitar concurrencia significa establecer un máximo explícito de trabajo simultáneo. Puede implementarse mediante:

- Worker pool fijo.
- Semáforo mediante channel buffered.
- Límite por proveedor.
- Límite por clase de request o modelo.

5.3. Backpressure

La backpressure es la forma en que un componente lento comunica indirectamente a los productores que no puede absorber más trabajo al mismo ritmo.

Opciones comunes:

1. Bloquear temporalmente al productor.
2. Mantener una cola limitada.
3. Rechazar nuevas requests.
4. Degradar funcionalidad.
5. Redirigir a otro backend.

Una cola ilimitada no elimina el problema: lo convierte en consumo creciente de memoria y latencia de cola.

5.4. CPU-bound frente a I/O-bound

Para tareas CPU-bound, un pool del orden del número de cores puede ser razonable. Para tareas I/O-bound, el número óptimo puede ser mucho mayor porque muchas goroutines están bloqueadas esperando.

En nuestro gateway, el pool no debe calcularse únicamente con `runtime.NumCPU()`. Debe basarse en restricciones externas: conexiones permitidas, cuotas del proveedor, latencia promedio, memoria y RPS objetivo.

6. Context, cancelación y deadlines

`context.Context` es una pieza central en servidores Go. Permite propagar cancelación, deadline y metadatos de request.

```
1 func callProvider(ctx context.Context, client *http.Client, url string) error {
2     ctx, cancel := context.WithTimeout(ctx, 8*time.Second)
3     defer cancel()
4
5     req, err := http.NewRequestWithContext(ctx, http.MethodPost, url, nil)
6     if err != nil {
7         return err
8     }
9
10    resp, err := client.Do(req)
11    if err != nil {
12        return err
13    }
14    defer resp.Body.Close()
15
16    return nil
17 }
```

Listing 2: Propagación de timeout hacia el proveedor

Si el cliente abandona la conexión, el contexto de la request puede cancelarse. Esa cancelación debe llegar hasta la llamada upstream para evitar seguir consumiendo GPU, tokens, sockets y memoria para una respuesta que nadie recibirá.

Regla práctica

Todo trabajo derivado de una request debería, salvo excepción justificada, estar ligado al contexto de esa request.

7. Servidor HTTP y networking

7.1. Ciclo request/response

En `net/http`, el servidor acepta conexiones, parsea requests y ejecuta un handler. El runtime y el paquete de red cooperan con el poller del sistema operativo para evitar dedicar un thread bloqueado a cada socket.

En Linux suele apoyarse en `epoll`; en macOS, en `kqueue`. La consecuencia práctica es que Go puede mantener un gran número de conexiones concurrentes sin crear un thread por conexión.

7.2. Timeouts críticos

Timeout	Qué protege
ReadHeaderTimeout	Clientes que envían headers extremadamente despacio
ReadTimeout	Lecturas de request excesivamente lentas
WriteTimeout	Respuestas que bloquean demasiado tiempo
IdleTimeout	Conexiones keep-alive inactivas
Upstream timeout	Proveedor LLM que no responde
Overall deadline	Tiempo total máximo de una operación

7.3. Cliente HTTP y connection pooling

Crear un `http.Client` nuevo por request impide aprovechar correctamente el pool de conexiones subyacente y puede aumentar DNS, TCP y TLS handshakes.

El gateway debe reutilizar clientes y configurar `http.Transport` con límites apropiados por host.

```
1 transport := &http.Transport{
2     MaxIdleConns: 200,
3     MaxIdleConnsPerHost: 100,
4     IdleConnTimeout: 90 * time.Second,
5 }
6
7 client := &http.Client{
8     Transport: transport,
9     Timeout: 15 * time.Second,
10 }
```

Listing 3: Cliente HTTP reutilizable

8. Rate limiting desde cero

8.1. Rate limiting vs concurrency limiting

No son equivalentes:

- **Rate limit:** limita cuántas operaciones se permiten por unidad de tiempo.
- **Concurrency limit:** limita cuántas operaciones pueden estar en vuelo simultáneamente.

Un cliente podría respetar 10 req/s pero mantener muchas requests simultáneas si cada una tarda 30 s.

8.2. Token Bucket

El estado mínimo del algoritmo es:

`{tokens, capacity, refillRate, lastRefill}`

A cada request se le exige un número de tokens. Los tokens se regeneran con el tiempo hasta alcanzar la capacidad máxima.

```
1 type TokenBucket struct {
2     mu sync.Mutex
3     capacity float64
```

```

4     tokens float64
5     refillRate float64
6     lastRefill time.Time
7 }
8
9 func (b *TokenBucket) Allow(now time.Time) bool {
10     b.mu.Lock()
11     defer b.mu.Unlock()
12
13     elapsed := now.Sub(b.lastRefill).Seconds()
14     b.tokens = math.Min(
15         b.capacity,
16         b.tokens+elapsed*b.refillRate,
17     )
18     b.lastRefill = now
19
20     if b.tokens < 1 {
21         return false
22     }
23
24     b.tokens--
25     return true
26 }

```

Listing 4: Esqueleto de Token Bucket concurrente

8.3. Refill lazy frente a ticker

Un refill mediante ticker por cliente es una mala arquitectura a gran escala: 100.000 clientes podrían implicar 100.000 timers/goroutines. El refill lazy calcula tokens acumulados únicamente cuando llega una request.

8.4. Claves del limiter

El rate limiting puede aplicarse por:

- API key.
- Usuario.
- IP.
- Modelo.
- Proveedor.
- Combinación de dimensiones.

El mapa de buckets debe tener política de limpieza para evitar crecimiento ilimitado de entradas inactivas.

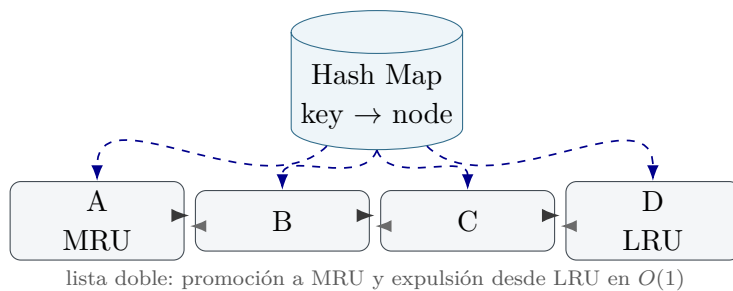
9. Cache In-Memory y LRU concurrente

9.1. Por qué una LRU usa mapa + lista doble

La LRU clásica combina:

- Hash map: localización $O(1)$ de una clave.

- Lista doblemente enlazada: promoción y expulsión $O(1)$.



9.2. Concurrency y locks

Aunque conceptualmente un `Get` parece una lectura, en una LRU tradicional también modifica la lista al promover el nodo a MRU. Por ello, un `RWMutex` no permite trivialmente que todos los Gets sean lectores puros.

Este detalle es importante: elegir `RWMutex` porque “hay muchas lecturas” puede ser incorrecto si dichas lecturas también actualizan metadatos.

9.3. TTL y tamaño

La capacidad no debería definirse solo por número de entradas. Una respuesta LLM puede variar entre unos bytes y varios megabytes. Una implementación robusta puede limitar:

- número de entradas;
- bytes totales;
- TTL por entrada;
- tamaño máximo de una única respuesta cacheable.

9.4. Cache key para LLM

Una clave correcta debe representar todos los parámetros que afectan semánticamente a la respuesta. Por ejemplo:

```

1 type CacheKeyInput struct {
2     Provider string
3     Model string
4     Prompt string
5     System string
6     Temperature float64
7     TopP float64
8     ToolsHash string
9 }

```

Listing 5: Modelo conceptual de clave de caché

Si omitimos `Temperature` o `System`, dos requests diferentes podrían colisionar lógicamente y devolver una respuesta incorrecta.

9.5. Singleflight y cache stampede

Escenario:

Request A -> cache miss -> llama a OpenAI
Request B -> cache miss -> llama a OpenAI
Request C -> cache miss -> llama a OpenAI

Si las tres requests son equivalentes, estamos pagando tres veces. La deduplicación de requests en vuelo permite que B y C esperen el resultado iniciado por A.

10. Routing, load balancing y health checking

10.1. Round Robin

Para una lista de N backends, un contador atómico puede seleccionar:

$$index = counter \bmod N$$

```
1 func nextIndex(counter *atomic.Uint64, n int) int {  
2     value := counter.Add(1)  
3     return int((value - 1) % uint64(n))  
4 }
```

Listing 6: Índice Round Robin con atomic

La dificultad real empieza cuando la lista cambia dinámicamente o cuando algunos backends están degradados.

10.2. Algoritmos comparados

Algoritmo	Ventaja	Limitación
Round Robin	Simple, barato y determinista	Ignora capacidad y latencia
Weighted RR	Representa capacidades distintas	Requiere pesos correctos
Random	Muy simple y sin estado global fuerte	Puede generar distribución desigual a corto plazo
Least Connections	Útil si las requests duran tiempos muy distintos	Mantiene estado de conexiones activas
Least Latency	Se adapta al rendimiento observado	Puede oscilar y requiere métricas estables
Consistent Hashing	Afinidad por clave y menor redistribución	No balancea bien cargas muy sesgadas sin técnicas adicionales

10.3. Health checks activos y pasivos

Un health checker activo realiza llamadas periódicas. Un check pasivo observa resultados reales de tráfico.

La combinación suele ser más robusta:

- Active: “¿el backend responde ahora?”
- Passive: “¿las requests reales están fallando?”

Para evitar flapping, conviene requerir varios fallos consecutivos antes de marcar unhealthy y varias recuperaciones antes de devolverlo a healthy.

11. Failover, retries, circuit breaker y bulkheads

11.1. Partial failure

En sistemas distribuidos, “UP” y “DOWN” son demasiado simplistas. Puede ocurrir:

Gateway	OK
Ollama	OK
OpenAI	DOWN
DNS	intermitente
Dashboard	OK
Network	degradada

El gateway debe seguir ofreciendo el máximo servicio posible sin ocultar degradaciones relevantes.

11.2. Retries

No todos los errores deben reintentarse. Un 400 por payload inválido no se resuelve repitiendo. Un timeout transitorio sí podría hacerlo.

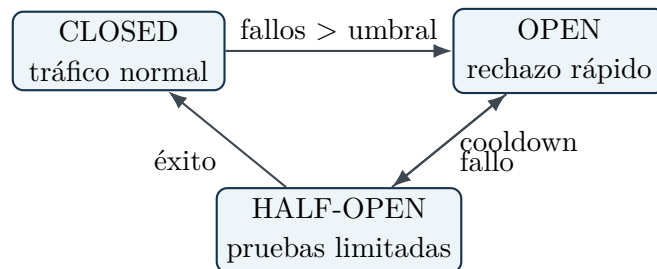
Una política típica combina exponential backoff y jitter:

$$wait_n = \min(maxDelay, base \cdot 2^n) + jitter$$

El jitter reduce la sincronización de muchos clientes que reintentan al mismo tiempo.

11.3. Circuit Breaker

Estados clásicos:



El health checker detecta estado; el circuit breaker protege el camino de ejecución evitando insistir sobre un backend ya conocido como problemático.

11.4. Bulkheads

El patrón bulkhead separa recursos para evitar que un fallo consuma toda la capacidad disponible. Por ejemplo:

- Pool de concurrencia para Ollama.
- Pool independiente para OpenAI.
- Límites separados por modelo.

Si OpenAI se degrada, sus requests no deberían ocupar todos los slots y bloquear también el tráfico que podría resolverse localmente.

12. Streaming, SSE y WebSockets

12.1. Streaming de respuestas LLM

En streaming, la latencia percibida depende especialmente de TTFT (Time To First Token), no solo de la latencia total.

Métricas útiles:

- TTFT.
- Tokens por segundo.
- Tiempo total.
- Bytes enviados.
- Abandono del cliente.

12.2. SSE vs WebSocket

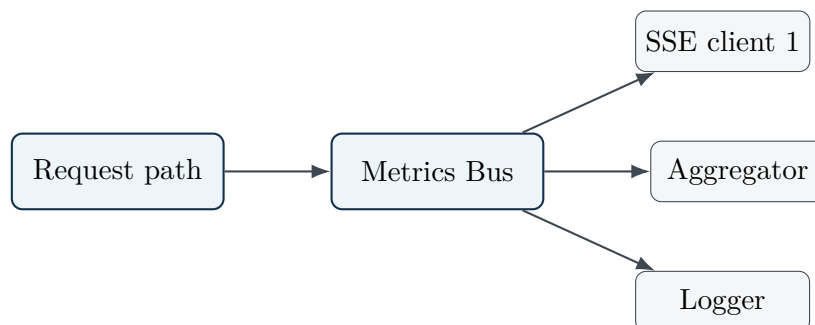
Aspecto	SSE	WebSocket
Dirección	Servidor → cliente	Bidireccional
Protocolo base	HTTP	Upgrade desde HTTP
Reconexión	Nativa/simple	Debe implementarse
Complejidad	Baja	Mayor
Ideal para	Métricas, eventos, logs	Control interactivo en tiempo real
Backpressure	Requiere política por consumidor	También requiere política explícita

Para el dashboard inicial, SSE suele ser suficiente porque el flujo principal es backend → frontend. WebSocket cobra sentido si el dashboard necesita comandos interactivos persistentes o comunicación bidireccional intensiva.

13. Pub/Sub interno y slow consumers

No queremos que la request principal se bloquee porque el dashboard sea lento.

Una arquitectura razonable es:



Si un consumidor se retrasa, debemos decidir entre bloquear, acumular, desconectar o descartar eventos. Para telemetría en vivo, descartar muestras antiguas suele ser mejor que bloquear el hot path.

14. Métricas y observabilidad

14.1. Tipos de métricas

- **Counter:** requests totales, errores, cache hits.
- **Gauge:** requests activas, tamaño de cola, workers ocupados.
- **Histogram:** distribución de latencias o tamaños.

14.2. Percentiles y tail latency

Una media puede ocultar degradaciones severas. Considérese:

$$P50 = 90 \text{ ms}, \quad P95 = 210 \text{ ms}, \quad P99 = 4.2 \text{ s}$$

El sistema parece rápido para la mayoría, pero 1 de cada 100 requests puede experimentar una latencia muy alta. En un gateway, esa cola larga puede venir de locks, GC, colas saturadas, conexiones nuevas, throttling del proveedor o retries.

14.3. Métricas del gateway

Métrica	Interpretación
Requests/s	Throughput de entrada y completado
Inflight requests	Trabajo simultáneo actual
Queue depth	Presión acumulada antes de procesamiento
P50/P95/P99 latency	Calidad de servicio y tail latency
Cache hit ratio	Efectividad de caché
Rate-limit rejects	Tráfico rechazado por política
Load-shed rejects	Tráfico rechazado por saturación
Provider latency	Latencia atribuible a cada backend
TTFT	Tiempo hasta el primer token
Tokens/s	Velocidad de generación
Failover count	Frecuencia de conmutaciones
Retry count	Presión extra causada por fallos transitorios
Estimated cost	Coste aproximado de proveedores externos
Estimated savings	Ahorro por caché o ejecución local
Health status	Estado de cada backend

14.4. Logs, métricas y traces

No son intercambiables:

- Métricas responden “¿está ocurriendo un patrón?”.
- Logs responden “¿qué ocurrió en este evento concreto?”.
- Traces responden “¿dónde se fue el tiempo a través de varios componentes?”.

Un `requestID` o `correlation ID` debe acompañar el flujo completo sin registrar prompts sensibles por defecto.

15. Memoria, GC y optimización

15.1. Hot path

El hot path del gateway puede representarse como:

HTTP parse

- > auth / rate limit
- > cache lookup
- > admission control
- > provider routing
- > upstream call
- > response / streaming
- > metrics

Cada asignación, lock o copia añadida aquí tiene potencial de multiplicarse por miles de requests.

15.2. sync.Pool

`sync.Pool` puede reutilizar objetos temporales como buffers, pero no debe usarse como caché semántica: el runtime puede vaciar el pool en cualquier momento.

15.3. False sharing

Dos contadores independientes pueden residir en la misma línea de caché de CPU. Si cores distintos escriben frecuentemente en ambos, el protocolo de coherencia puede provocar tráfico innecesario. Este fenómeno se denomina false sharing.

Es un tema de optimización avanzada y debe medirse antes de introducir padding manual.

15.4. Profiling

Herramientas relevantes:

- `go test -bench=. -benchmem`
- CPU profile
- Heap profile
- Mutex profile
- Block profile
- Goroutine profile
- pprof

Regla de rendimiento

No optimizar una estructura porque “parece lenta”. Primero identificar el cuello de botella con benchmark o profile, cambiar una variable, y volver a medir.

16. Little’s Law, throughput y saturación

Little’s Law relaciona número promedio de elementos en el sistema L , tasa de llegada λ y tiempo promedio W :

$$L = \lambda W$$

Si el gateway procesa 500 req/s y cada request permanece 2 s en el sistema, hay aproximadamente:

$$L = 500 \times 2 = 1000$$

requests en vuelo en promedio.

Esto explica por qué latencia y concurrencia están íntimamente ligadas. Si el proveedor se ralentiza de 2 s a 10 s manteniendo la misma llegada, el sistema necesitaría soportar aproximadamente 5.000 requests en vuelo para no rechazar tráfico, con el consiguiente aumento de memoria, sockets y presión interna.

17. Seguridad y límites defensivos

Un gateway expuesto debe asumir inputs maliciosos o simplemente incorrectos.

Controles mínimos:

- Tamaño máximo de body.
- Validación estricta de JSON.
- API keys y secretos fuera del repositorio.
- TLS en despliegue real.
- No loggear Authorization headers.
- No loggear prompts completos por defecto.
- CORS explícito.
- Timeouts en servidor y upstream.
- Límites de concurrencia y rate limiting.
- Sanitización de logs enviados al dashboard.

18. Frontend React y dashboard en tiempo real

El dashboard no debería ser un simple ejercicio visual; debe actuar como consola operativa.

18.1. Paneles recomendados

- RPS actual y histórico.
- P50/P95/P99.
- Cache hit ratio.
- Coste estimado y ahorro acumulado.
- Estado de cada backend.
- Requests activas y profundidad de cola.
- Failovers y retries.
- TTFT y tokens/s.
- Live logs con filtros por nivel, proveedor y request ID.

18.2. Evitar re-renders excesivos

Si recibimos 1.000 eventos/s, no debemos forzar 1.000 renders/s. El frontend puede:

- Agregar muestras en ventanas temporales.
- Mantener ring buffers.
- Actualizar charts a intervalos de 250–1000 ms.
- Separar estado crudo de estado derivado.
- Memoizar componentes donde tenga sentido.

19. Estructura recomendada del monorepo

```
ai-optimized-gateway-proxy/  
|-- backend/  
|   |-- cmd/  
|   |   '-- gateway/  
|   |       '-- main.go  
|   |-- internal/  
|   |   |-- httpapi/  
|   |   |-- middleware/  
|   |   |-- admission/  
|   |   |-- ratelimit/  
|   |   |-- cache/  
|   |   |-- provider/  
|   |   |   |-- openai/  
|   |   |   '-- ollama/  
|   |   |-- routing/  
|   |   |-- health/  
|   |   |-- resilience/  
|   |   |-- metrics/  
|   |   |-- realtime/  
|   |   '-- config/  
|   |-- pkg/  
|   |-- tests/  
|   |-- go.mod  
|   '-- go.sum  
|-- frontend/  
|   |-- src/  
|   |   |-- components/  
|   |   |-- features/  
|   |   |-- hooks/  
|   |   |-- services/  
|   |   |-- types/  
|   |   '-- pages/  
|   |-- package.json  
|   '-- tsconfig.json  
|-- deploy/  
|   |-- docker/  
|   '-- compose/
```

```
|-- docs/
|   |-- architecture/
|   |-- adr/
|   '-- benchmarks/
|-- scripts/
|-- .github/
|   '-- workflows/
|-- Makefile
'-- README.md
```

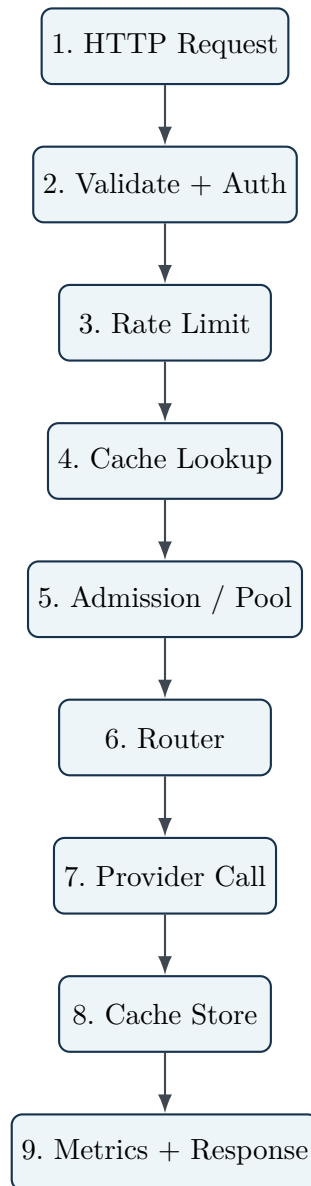
19.1. Por qué esta estructura

`cmd/gateway` contiene el `composition root`: crea dependencias y arranca el proceso. La lógica de negocio se concentra en `internal`, evitando que otros módulos externos importen accidentalmente detalles internos.

Los paquetes deben nacer de responsabilidades reales, no de patrones abstractos. En Go, introducir una interface antes de existir dos implementaciones o una necesidad clara de sustitución puede ser sobreingeniería.

20. Flujo completo de una request

Una request típica podría recorrer:



Este orden puede cambiar por decisiones de producto. Por ejemplo, consultar caché antes del rate limiter puede permitir hits baratos incluso cuando el proveedor está saturado, pero también puede crear una vía para eludir una política de cuota si el contrato exige contabilizar todas las requests. Esta es una decisión de diseño, no una verdad universal.

21. Ejemplo práctico integrado

Supongamos una configuración:

- Ollama local: 20 requests concurrentes sostenibles.
- OpenAI: 100 req/s de cuota.
- Cache LRU: 1 GB máximo, TTL 15 min.
- Worker/admission queue: 500 elementos.
- Rate limit por API key: 10 req/s, burst 20.

Llega una request:

1. El servidor valida body y autenticación.

2. Token Bucket concede un token.
3. Se calcula la cache key incluyendo modelo y parámetros de generación.
4. Hay miss.
5. Se consulta singleflight: nadie está procesando esa clave.
6. Admission control reserva capacidad.
7. Router selecciona Ollama porque está healthy y tiene capacidad.
8. El contexto impone un deadline de 8 s.
9. Ollama falla por conexión.
10. Health/passive checker registra fallo.
11. La política determina que es retryable.
12. Se intenta OpenAI, respetando su propio concurrency limit.
13. OpenAI responde correctamente.
14. Se guarda la respuesta si cumple política de caché.
15. Se liberan recursos del admission control.
16. Se emiten métricas: latency, provider, failover=1, tokens, cost, cache miss.
17. El dashboard recibe un evento agregado.

La clave está en que cada componente protege una dimensión distinta del sistema. Rate limiting no sustituye admission control; health checking no sustituye circuit breaking; caching no sustituye singleflight; retries sin deadlines pueden empeorar una caída.

22. Testing y verificación de corrección

22.1. Unit tests

Ejemplos prioritarios:

- Token bucket con reloj controlable.
- LRU: promoción, eviction y TTL.
- Round Robin con backends healthy/unhealthy.
- Circuit breaker y transiciones de estado.
- Policy de retry por tipo de error.

22.2. Race detector

```
go test -race ./...
```

Debe formar parte del CI. No demuestra ausencia de todos los bugs de concurrencia, pero detecta accesos de memoria sin sincronización durante las ejecuciones cubiertas.

22.3. Load testing

Un test útil incrementa progresivamente carga y observa el punto de saturación. Métricas a correlacionar:

- RPS.
- P95/P99.
- queue depth.

- heap.
- número de goroutines.
- CPU.
- mutex/block profile.
- errores y rechazos.

El objetivo no es únicamente encontrar el máximo RPS, sino comprender **cómo falla** el sistema cuando supera su capacidad.

23. CI/CD y despliegue

Pipeline recomendado:

1. Format y lint.
2. Unit tests.
3. Race detector.
4. Integration tests.
5. Build backend/frontend.
6. Benchmarks de referencia en ramas controladas.
7. Docker build.
8. Security scan opcional.

Para Docker, un multi-stage build permite compilar el binario Go en una etapa y copiar únicamente el ejecutable a una imagen final mínima.

24. Roadmap de aprendizaje por hitos

Milestone	Entregable	Conceptos principales
M0	Monorepo y servidor HTTP mínimo	net/http, config, graceful shutdown, estructura de paquetes
M1	Concurrencia controlada	goroutines, channels, worker pool, semáforos, context
M2	Token Bucket	mutex, tiempo monotónico, rate limiting, limpieza de estado
M3	Provider abstraction	interfaces, adapters, HTTP clients, OpenAI/Ollama
M4	Routing y health	atomics, Round Robin, health checks, failover
M5	Resiliencia	retries, backoff, jitter, circuit breaker, bulkheads
M6	Cache LRU	hashmap + lista doble, locks, TTL, memory limits
M7	Singleflight	in-flight deduplication, cache stampede
M8	Streaming	SSE upstream/downstream, TTFT, cancellation
M9	Telemetría	counters, gauges, histograms, pub/sub

M10	React dashboard	realtime state, charts, aggregation, reconnection
M11	Performance	benchmarks, pprof, allocation y lock profiling
M12	Production hardening	Docker, CI, race tests, load tests, docs de arquitectura

25. Errores de diseño que el proyecto debe evitar

1. Cola de jobs ilimitada.
2. Crear un `http.Client` por request.
3. Un ticker/goroutine por bucket de rate limit.
4. Cache LRU sin límite de bytes.
5. Reintentar todos los errores.
6. Retries sin jitter.
7. Health checker que cambia estado con un único fallo.
8. Un mutex global protegiendo trabajo lento.
9. Emitir telemetría bloqueante en el hot path.
10. Ignorar cancelación del cliente.
11. Medir únicamente latencia media.
12. Optimizar antes de perfilar.
13. Introducir interfaces y capas sin necesidad concreta.

26. Criterios de calidad de nivel Mid/Senior

Un equipo técnico valorará más el razonamiento que la cantidad de features. El proyecto debería permitir explicar con precisión:

- por qué existe cada límite de concurrencia;
- cómo evita crecimiento ilimitado de memoria;
- qué garantías de thread-safety ofrece cada estructura;
- qué errores son retryable y por qué;
- cómo se degrada ante caída parcial de proveedores;
- cómo se evita que observabilidad bloquee producción;
- cómo se mide el rendimiento real;
- qué trade-offs se aceptaron conscientemente.

Documentar estas decisiones como ADRs (Architecture Decision Records) convertirá el repositorio en una demostración de criterio, no únicamente de implementación.

27. Conclusión

`ai-optimized-gateway-proxy` es un excelente vehículo para aprender ingeniería de sistemas porque fuerza a conectar conceptos que normalmente se estudian aislados. Goroutines, channels y mutexes solo adquieren sentido cuando se relacionan con colas, sockets, límites externos y fallos parciales. Un algoritmo LRU deja de ser un ejercicio académico cuando 20.000 requests compiten por el mismo

lock. Un retry deja de ser una “buena práctica” cuando multiplica la carga durante una caída. Una media de latencia deja de ser útil cuando el P99 revela saturación.

El diseño final debe perseguir cuatro propiedades:

1. **Corrección:** ausencia de data races, invariantes claras y respuestas semánticamente correctas.
2. **Control:** límites explícitos de memoria, concurrencia, cola, tiempo y frecuencia.
3. **Resiliencia:** degradación controlada ante fallos parciales, timeouts y proveedores lentos.
4. **Observabilidad:** capacidad de explicar qué ocurre internamente mediante métricas, logs, traces y profiling.

La implementación debe avanzar por milestones pequeños. El primer paso técnico recomendado es construir un servidor HTTP nativo mínimo con configuración y graceful shutdown, medir su comportamiento y, a partir de ahí, introducir bounded concurrency. Este orden permite que cada nueva abstracción aparezca como respuesta a un problema observable, que es precisamente la forma más sólida de desarrollar criterio de ingeniería.